```
from google.colab import files
uploaded = files.upload()
```

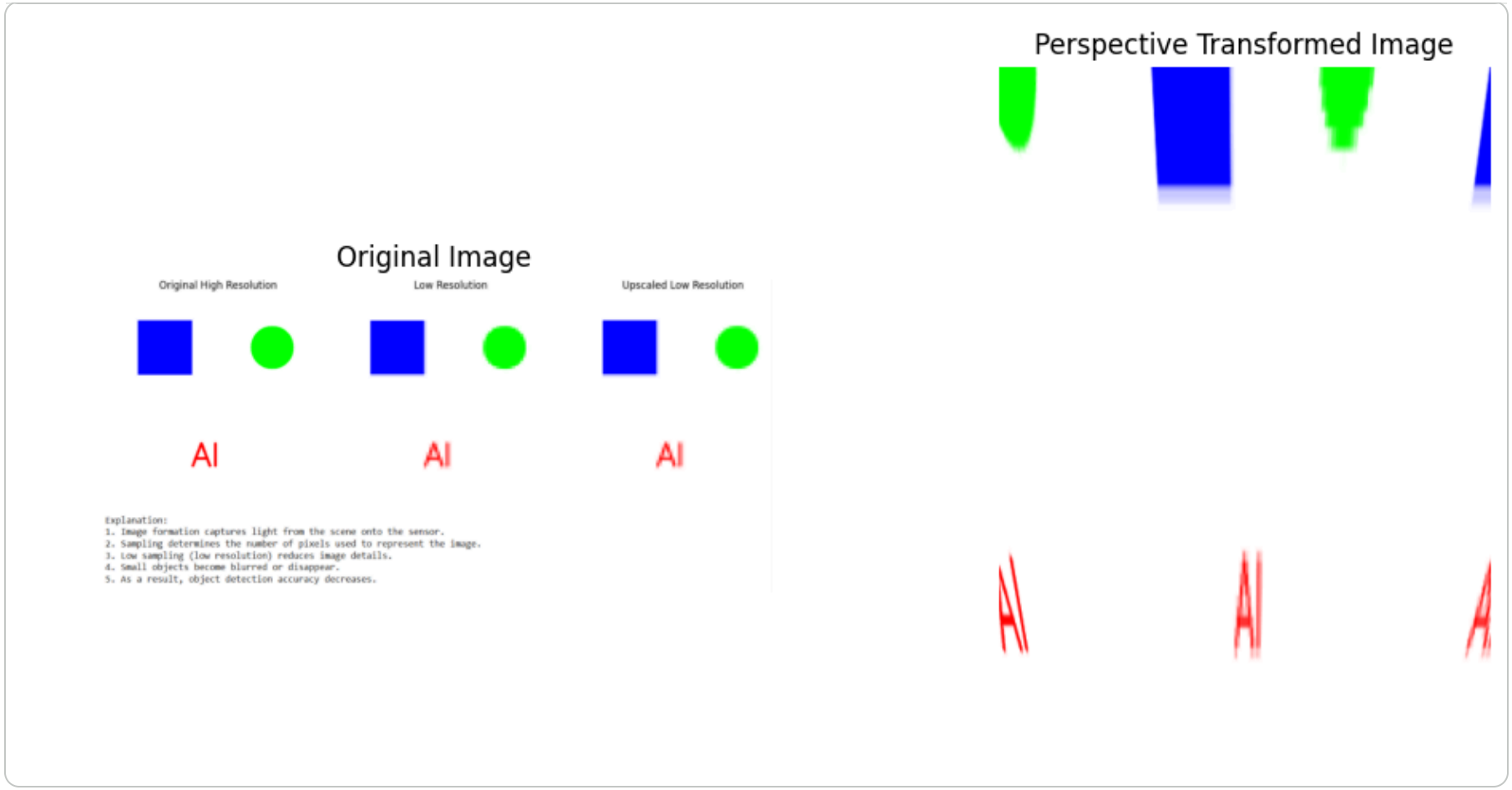
Choose files Screenshot … 094656.png

Screenshot 2026-07-17 094656.png(image/png) - 65312 bytes, last modified: 17/07/2026 - 100% done

```
Saving Screenshot 2026-07-17 094656.png to Screenshot 2026-07-17 094656.png
```

```
import cv2
import numpy as np
import matplotlib.pyplot as plt

# Use the recently uploaded image
image_path = 'Screenshot 2026-07-17 094656.png'

img = cv2.imread(image_path)

if img is None:
    print(f"Error: Could not load image from {image_path}. Please make sure the file exists and is ac
else:
    rows, cols = img.shape[:2]

    # Define the ROI points from the original problem (adjusting based on typical image dimensions)
    # The roi_points should be 4 points from the source image that define the region to transform
    # The target_points define how these 4 points will map to the new image
    # For a typical image, roi_points should be within its bounds.
    # Let's assume the user wants to transform a rectangle within the image to a new trapezoidal shap
    # For demonstration, let's pick 4 points that represent a central region of the image.
    # Original points for perspective transformation (assuming they are from the source image)
    roi_points = np.float32([[cols * 0.25, rows * 0.25], [cols * 0.75, rows * 0.25], [cols * 0.75, ro

    # Target points for perspective transformation (these define the shape after transformation)
    # These values were hardcoded to (400,600) output size which might not be ideal for arbitrary ima
    # Let's make them relative or define a fixed output size if needed.
    # For simplicity, let's project the square ROI to a trapezoid within the same output dimension
    target_width = 400 # Example fixed width for output
    target_height = 600 # Example fixed height for output
```

```
# Adjust target points to avoid creating overly large or small output based on the image size.
# The original target_points were likely intended for a specific application where the output dim
# For a general example, let's map the roi_points to a full output image, or a specific region of
# For now, let's map them to a new, smaller trapezoid within a hypothetical output space.
target_points = np.float32([[0, 0], [target_width, 0], [target_width * 0.8, target_height], [targ

# Get the perspective transformation matrix
M = cv2.getPerspectiveTransform(roi_points, target_points)

# Apply the perspective transformation
# The output size should be chosen carefully. (target_width, target_height) is a common choice.
perspective_img = cv2.warpPerspective(img, M, (target_width, target_height))

# Convert images from BGR to RGB for correct display with matplotlib
img_rgb = cv2.cvtColor(img, cv2.COLOR_BGR2RGB)
perspective_img_rgb = cv2.cvtColor(perspective_img, cv2.COLOR_BGR2RGB)

# Display the original and transformed images using matplotlib
plt.figure(figsize=(12, 6))

plt.subplot(1, 2, 1)
plt.imshow(img_rgb)
plt.title('Original Image')
plt.axis('off')

plt.subplot(1, 2, 2)
plt.imshow(perspective_img_rgb)
plt.title('Perspective Transformed Image')
plt.axis('off')

plt.show()

# If you want to save the transformed image, you can use cv2.imwrite
# cv2.imwrite('Perspective_Transformed_Image.jpg', perspective_img)
```


Perspective Transformed Image
Original Image
Original High Resolution
Low Resolution
Upscaled Low Resolution
AI
AI
AI
Explanation:
1. Image formation captures light from the scene onto the sensor.
2. Sampling determines the number of pixels used to represent the image.
3. Low sampling (low resolution) reduces image details.
4. Small objects become blurred or disappear.
5. As a result, object detection accuracy decreases.